Banking API Backend: Study Guide

Oct 1, 2026 · @Duong Lam

1. Overview

The backend is a REST API for a simple bank. Customers open accounts, move money between them, and every money movement is recorded in an audit trail. It is written in Python with FastAPI and stores its data in MongoDB Atlas.

Every file is explained line by line in the `Line by line` tab.

What it can do

- **Customers:** create, read, update and delete them (CRUD).
- **Accounts:** full CRUD. One customer can own many accounts. The balance only changes through a deposit, withdrawal or transfer.
- **Premium accounts:** list every account whose balance is at or above a threshold you pass in.
- **Transfers:** move money from one account to another.
- **Audit trail:** trace any money movement: who did it, when, which accounts, and how much.

Tech stack

Piece	What it is	Why it's in the project
Python 3.12	The language	Everything is written in it
FastAPI	Web framework	Turns Python functions into HTTP routes, validates input, and generates Swagger docs at `/docs` for free
Pydantic v2	Data models	Checks every request body and shapes every response
Uvicorn	Web server	Runs the app: `python -m uvicorn main:app --reload`
MongoDB Atlas	Cloud NoSQL database	Stores customers, accounts and transactions as documents

Piece	What it is	Why it's in the project
PyMongo	MongoDB driver	Lets Python talk to Atlas
python-dotenv	Config loader	Reads the database password from `backend/.env`, so it's never in the code or on GitHub
Swagger, Postman	Testing tools	Manual testing (Swagger) and an automated collection (Postman)
Git, GitHub	Version control	History from the in-memory version to the MongoDB version

Every file and its job

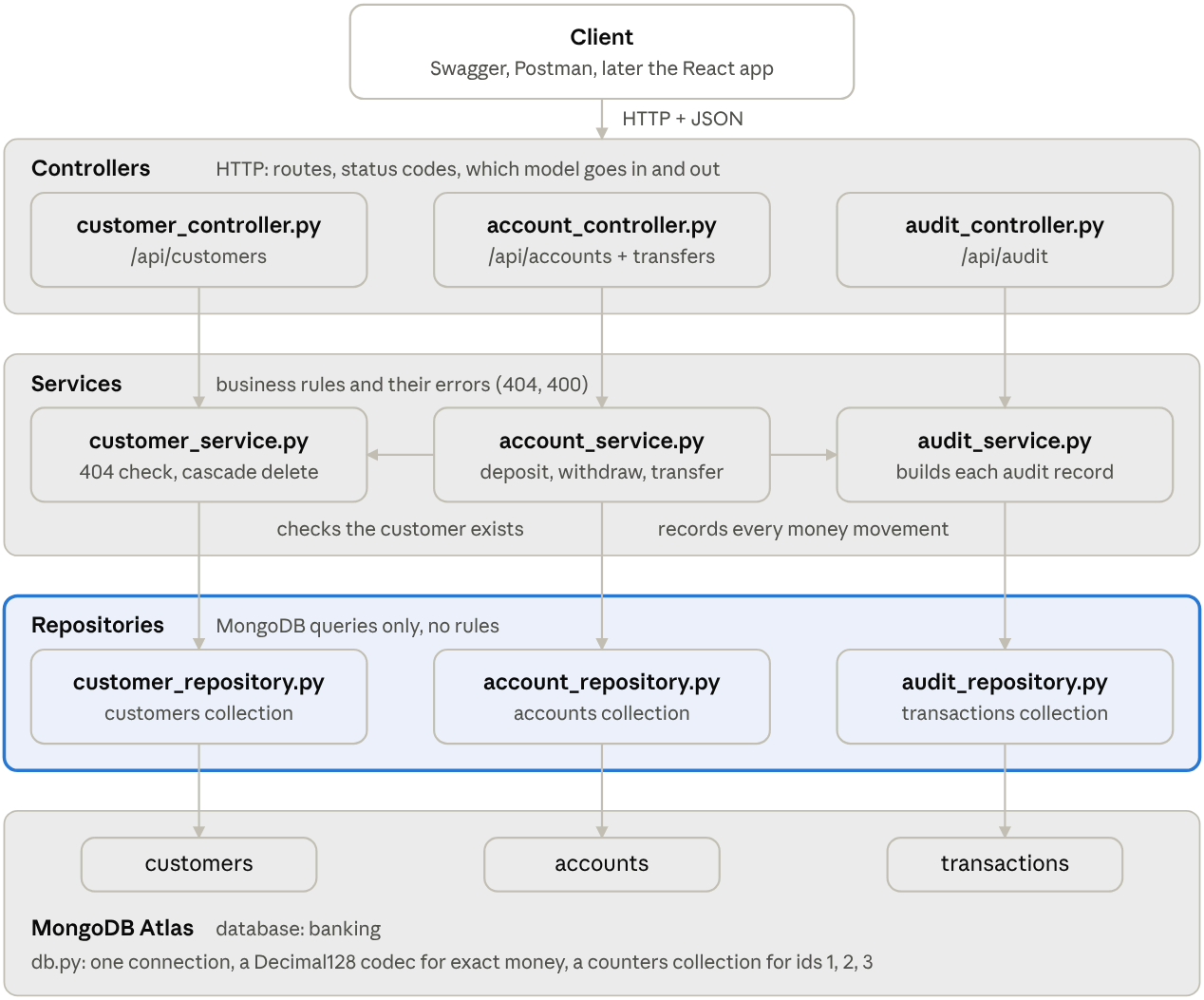
File	Layer	Job
`main.py`	Entry point	Creates the app and registers the routers
`dependencies.py`	Wiring	Creates every repository and service once and connects them
`db.py`	Database	Connects to Atlas, stores money exactly, hands out 1, 2, 3 ids
`models.py`	Models	The shape of every request and response
`customer_controller.py`, `account_controller.py`, `audit_controller.py`	Controller	The HTTP routes
`customer_service.py`, `account_service.py`, `audit_service.py`	Service	The business rules
`customer_repository.py`, `account_repository.py`, `audit_repository.py`	Repository	The MongoDB queries
`requirements.txt`	Setup	The packages to install

File	Layer	Job
`.env`	Secret config	The Atlas connection string (git ignores it)
`test_mongo.py`	Utility	A one-off check that Atlas is reachable

2. The layered design

The code is split into three layers so each file has exactly one job: controllers handle HTTP, services hold the business rules, and repositories talk to MongoDB. Calls only go one way, downward: controller to service to repository to database.

Requests flow down three layers, and only repositories touch MongoDB



models.py: the data shapes every layer shares
dependencies.py creates each object once and connects them; main.py registers the routers

backend architecture · 3 layers, 9 files, 3 collections

Each request enters at the top and only moves down. `AccountService` is the one service that calls the other two: it checks the customer exists and records every money movement.

Layer	Its job	It never	Example
Controller	URLs, HTTP methods, status codes, which model goes in and out	Checks a rule or touches the database	The deposit route is one line: `return service.deposit(account_id, data.amount)`
Service	Business rules and their errors (404, 400)	Writes a database query	"You can't withdraw more than the balance" lives only in `AccountService`
Repository	Reads and writes one MongoDB collection	Decides anything; it returns `None` instead of raising a 404	`find_premium` runs `{"balance": {"$gte": threshold}}`
Models	The shape of the data, shared by every layer	Contains logic	`Account` has `id`, `customerId`, `accountType`, `balance`

Why build it this way

- **You can swap the storage without touching the rules.** This already happened: the app started with in-memory Python dictionaries, and moving to MongoDB changed only the repositories and `dependencies.py`. Git shows it (commit `c7e0962` in-memory, `60efed3` MongoDB).
- **Each rule lives in one place.** If the overdraft rule changes, you edit one method, and every route that uses it follows.
- **Each layer can be explained and tested on its own.** A bug in a query is in a repository; a wrong error message is in a service.

How it maps to MVC (the assignment grades "clean MVC separation"):

- **Model:** `models.py` for the data shapes, plus the repositories that store them.
- **View:** the JSON responses today, and the React frontend later.
- **Controller:** the `*_controller.py` files.
- The **service layer** is the extra layer between controller and model that keeps business logic out of both. Most real backends (Spring Boot, .NET, NestJS) use this same controller / service / repository split.

3. One request end to end: a deposit

A deposit passes through every layer and writes to two collections, accounts and transactions, so it is the best single request to walk through in a presentation. The client sends `POST /api/accounts/1/deposit` with the body `{"amount": 500}`.

1. **Uvicorn** receives the HTTP request and hands it to the FastAPI `app` created in `main.py`.
2. **FastAPI matches the route** `@router.post("/{account_id}/deposit")` in `account_controller.py`. It converts `"1"` from the URL into the `int` `account_id`, and checks the body against `AmountIn`. A body like `{"amount": "abc"}` is rejected here with **422**, before any of our code runs.
3. **The controller** makes one call: `service.deposit(account_id, data.amount)`.
4. **`AccountService.deposit`** first calls `get_account(1)`, which asks the repository for the account. The repository runs `find_one({"_id": 1})`. No document means `None`, and the service turns that into **404 "Account not found"**.
5. **Rule check:** an amount of 0 or less raises **400 "Amount must be positive"**.
6. **The new balance** is `account.balance + amount`. Both are `Decimal`, so the maths is exact.
7. **`AccountRepository.update_balance`** runs `find_one_and_update({"_id": 1}, {"$set": {"balance": new_balance}}, return_document=AFTER)`. The codec in `db.py` turns the `Decimal` into Mongo's `Decimal128`. Mongo returns the updated document, and `from_doc` turns it into an `Account`.
8. **The audit record:** the service calls `self._audit.record(TxnType.DEPOSIT, amount, account.customer_id, to_account_id=1)`. `AuditService` stamps the time in UTC, and `AuditRepository` gets the next id from the `counters` collection and inserts the record into `transactions`.
9. **The response:** the service returns the `Account`. Because the route says `response_model=Account`, FastAPI converts `customer_id` to `customerId` and the `Decimal` to a JSON number, and sends **200**:

```
{"id": 1, "customerId": 1, "accountType": "SAVINGS", "balance": 500.0}
```

The point to make out loud: the controller never touched MongoDB, the repository never checked a rule, and the service never parsed JSON. Each layer did one thing.

4. models.py: the shape of the data

`models.py` defines every request and response as a Pydantic class. FastAPI uses these classes to reject bad input with a 422 and to turn Python objects into JSON on the way out.

The base class: `CamelModel`

```
class CamelModel(BaseModel):
    model_config = ConfigDict(alias_generator=to_camel,
populate_by_name=True)
```

- `alias_generator=to_camel` gives every field a camelCase name for JSON. Python writes `customer_id`; the API sends and accepts `customerId`, which is what JavaScript and React expect.
- `populate_by_name=True` means Python code can still build models with the snake_case names.
- Every model inherits from `CamelModel`, so this setting is written once and applies everywhere.

Money: `Decimal`, never `float`

```
Money = Annotated[Decimal, PlainSerializer(float, return_type=float,
when_used="json")]
```

- A `float` can't store most decimal numbers exactly: in Python, `0.1 + 0.2` gives `0.30000000000000004`. That is unacceptable for a bank.
- `Decimal` does exact base-10 maths, so every balance and amount is a `Decimal` inside the app.
- The `PlainSerializer` converts it to a JSON number only at the very end, so the client still receives `500.0`.

The In / Out pattern

Each thing has one model for what the client *sends* and one for what the API *returns*. The returned model inherits the input model and adds what only the server may set.

Model	Used for	Fields
`CustomerIn`	Create and update body	`name`, `email`

Model	Used for	Fields
`Customer(CustomerIn)`	Response	inherited + `id`
`AccountIn`	Create body	`customerId`, `accountType`
`AccountUpdate`	PUT body	`accountType` only
`Account`	Response	`id`, `customerId`, `accountType`, `balance` (defaults to 0.00)
`AmountIn`	Deposit and withdraw body	`amount`
`TransferIn`	Transfer body	`fromAccountId`, `toAccountId`, `amount`
`TransactionIn`	Built by the audit service	`type`, `amount`, `customerId`, `fromAccountId`, `toAccountId`, `timestamp`
`Transaction(TransactionIn)`	Audit response	inherited + `id`

- A client **can't choose an id**, because no input model has one.
- A client **can't set a balance**. `AccountIn` has no balance, so new accounts start at 0. `AccountUpdate` has only the account type, so a PUT can't change money. If someone sends `"balance": 99999` in a PUT, Pydantic simply ignores it.

Enums: a fixed list of allowed values

- `AccountType(str, Enum)` allows only `SAVINGS` and `CHECKING`. Anything else, like `"GOLD"`, gets a 422, and Swagger shows a dropdown.
- `TxnType(str, Enum)` is `DEPOSIT`, `WITHDRAW` or `TRANSFER`.
- Inheriting from `str` as well as `Enum` makes the value behave like a normal string, so it is saved to MongoDB and sent as JSON as plain text.

Optional fields

In `TransactionIn`, `from_account_id: int | None = None` means the field may be empty. A deposit has no source account, and a withdrawal has no destination, so one of the two is always `None`.

`AccountIn.example()` is a `@staticmethod` that returns a sample account. The API doesn't use it; it shows the static method concept.

5. db.py and the data in MongoDB

`db.py` is the only file that knows how to reach the database. It also solves three problems MongoDB brings with it: storing money exactly, giving out simple numbered ids, and Mongo calling the id `_id`.

Connecting

```
load_dotenv()
client = MongoClient(os.environ["MONGODB_URI"])
db = client.get_database(os.getenv("MONGODB_DB", "banking"),
codec_options=_options)
```

- `load_dotenv()` reads `backend/.env` into environment variables, so the password lives in a file that `.gitignore` keeps off GitHub.
- One `MongoClient` is created for the whole app. It keeps a pool of connections and reuses them for every request.
- The database is called `banking`, unless the `MONGODB_DB` variable says otherwise.

Problem 1: money precision. Fix: a codec

MongoDB's exact decimal type is `Decimal128`; Python's is `Decimal`. PyMongo refuses a plain `Decimal` ("cannot encode object"). `DecimalCodec` teaches it both directions:

```
class DecimalCodec(TypeCodec):
    python_type = Decimal
    bson_type = Decimal128
    def transform_python(self, value): return Decimal128(value)   # Python ->
Mongo
    def transform_bson(self, value): return value.to_decimal()     # Mongo ->
Python
```

It is registered once in the database's `codec_options`, so every read and write converts automatically. No other file ever thinks about it. `tz_aware=True` in the same options makes timestamps come back with their UTC time zone attached.

Problem 2: simple ids. Fix: a counters collection

MongoDB has no `AUTO_INCREMENT`. Its default ids look like `66fa3b2e9c...`. To keep URLs like `/api/accounts/1`, `next_id` keeps one counter per collection:

```
def next_id(db, name):
    counter = db.counters.find_one_and_update(
        {"_id": name}, {"$inc": {"seq": 1}}, upsert=True,
return_document=ReturnDocument.AFTER)
    return counter["seq"]
```

- `$inc` adds 1 to `seq` inside the database, in one step. Two requests at the same moment can never get the same number.
- `upsert=True` creates the counter the first time it's used.
- `ReturnDocument.AFTER` returns the value *after* adding 1.

Problem 3: `_id` vs `id`. Fix: `from_doc`

Mongo stores the id as `_id`, but the models call it `id`. `from_doc(Account, doc)` copies the document, renames `_id` to `id`, and builds the model. Given `None`, it returns `None`.

The four collections

Collection	Example document	Notes
`customers`	`{"_id": 1, "name": "John Doe", "email": "john@example.com"}`	One per customer
`accounts`	`{"_id": 1, "customer_id": 1, "account_type": "SAVINGS", "balance": Decimal128("1000.00")}`	`customer_id` points at its owner
`transactions`	`{"_id": 4, "type": "TRANSFER", "amount": 300, "customer_id": 1, "from_account_id": 1, "to_account_id": 3, "timestamp": ISODate(...)}`	The audit trail, never edited
`counters`	`{"_id": "accounts", "seq": 7}`	The next-id counters

How the relationships work

- **One customer, many accounts:** each account stores its owner's id in `customer_id`, like a foreign key in SQL.
- **Transactions point at accounts** through `from_account_id` and `to_account_id`.
- MongoDB doesn't enforce these links the way SQL foreign keys do. The service layer does instead: you can't create an account for a customer that doesn't exist.

- **Why reference instead of embedding** the accounts inside the customer document: accounts change on their own (every deposit) and are searched on their own (premium accounts). Embedding would make both harder.

6. Repositories: the MongoDB queries

Each repository wraps one collection and only translates between models and Mongo documents. None of them contains a business rule, and none raises a 404: when something isn't found, they return `None` and let the service decide.

The shared pattern

```
class AccountRepository:
    def __init__(self, db: Database):
        self._db = db               # needed for next_id
        self._col = db.accounts     # the collection, like a SQL table
```

Every method follows the same three steps: build a filter, run one Mongo call, convert the result with `from_doc`.

Every query, with its SQL equivalent

Repository method	MongoDB call	SQL equivalent
`CustomerRepository.find_all`	`find()`	`SELECT * FROM customers`
`CustomerRepository.find_by_id`	`find_one({"_id": id})`	`SELECT * FROM customers WHERE id = ?`
`CustomerRepository.create`	`next_id` + `insert_one(doc)`	`INSERT INTO customers ...`
`CustomerRepository.update`	`find_one_and_update({"_id": id}, {"$set": {...}}, AFTER)`	`UPDATE customers SET name = ?, email = ? WHERE id = ?`
`CustomerRepository.delete`	`delete_one({"_id": id})`	`DELETE FROM customers WHERE id = ?`
`AccountRepository.find_by_customer_id`	`find({"customer_id": id})`	`SELECT * FROM accounts WHERE customer_id = ?`

Repository method	MongoDB call	SQL equivalent
`AccountRepository.find_premium`	`find({"balance": {"$gte": threshold}})`	`SELECT * FROM accounts WHERE balance >= ?`
`AccountRepository.create`	`insert_one` with `balance: 0.00`	`INSERT INTO accounts ...`
`AccountRepository.update_balance`	`find_one_and_update(..., {"$set": {"balance": b}})`	`UPDATE accounts SET balance = ? WHERE id = ?`
`AccountRepository.delete_by_customer_id`	`delete_many({"customer_id": id})`	`DELETE FROM accounts WHERE customer_id = ?`
`AuditRepository.find_all`	`find().sort("_id", -1)`	`SELECT * FROM transactions ORDER BY id DESC`
`AuditRepository.find_by_account_id`	`find({"$or": [{"from_account_id": id}, {"to_account_id": id}]})`	`... WHERE from_account_id = ? OR to_account_id = ?`

The account repository also has `find_all`, `find_by_id`, `update` and `delete` in the same shape as customers. The audit repository has `create`, `find_all`, `find_by_id` and `find_by_account_id`, and nothing that changes a record.

Details worth explaining

- **`$set`** changes only the fields listed. Replacing the whole document instead would mean an account type update could wipe the balance.
- **`ReturnDocument.AFTER`** makes `find_one_and_update` return the document *after* the change. The default returns the old version.
- **`find_premium` filters inside the database.** Only matching accounts travel over the network. Loading every account into Python and looping would get slower as the bank grows.
- **`delete_many`** removes every account a customer owns in one call. That single line is the cascade delete.
- **`$or`** finds transactions where the account sent money *or* received it, so a transfer shows up in both accounts' history.
- **`.sort("_id", -1)`** puts the newest records first, since ids only go up.

- **Small helpers:** `_doc_to_account` is a module-level function and `_doc_to_transaction` is a `@staticmethod`, because neither needs `self`. The leading underscore says "internal, don't call from outside".
- **The audit repository has no update or delete method.** That is how the code keeps the audit trail append-only.

7. Services: the business rules

Services hold every business rule, and each rule lives in exactly one place. They are the only layer that decides whether a request is allowed, and they report problems with an HTTP error code.

Every rule, where it lives, and what the client gets

Rule	Method	Response when broken
The customer must exist	`CustomerService.get_customer`	404 "Customer not found"
The account must exist	`AccountService.get_account`	404 "Account not found"
An account needs a real owner	`create_account` (calls `get_customer`)	404
The amount must be above 0	`deposit`, `withdraw`, `transfer`	400 "Amount must be positive"
No withdrawing more than the balance	`withdraw`, `transfer`	400 "Insufficient funds"
No transfer to the same account	`transfer`	400 "Cannot transfer to the same account"
The premium threshold can't be negative	`get_premium_accounts`	400
The transaction must exist	`AuditService.get_transaction`	404 "Transaction not found"
Deleting a customer deletes their accounts	`delete_customer`	(not an error: cascade)
Every money movement is recorded	`deposit`, `withdraw`, `transfer`	(not an error: audit)

The three error codes mean different things: **404** = what you asked for doesn't exist, **400** = it exists but the rule says no, **422** = your request was malformed. Pydantic produces the 422s automatically, before the service is even called.

`CustomerService(repo, account_repo)`

- `get_customer` is the single 404 check. `update_customer`, `delete_customer` and even `AccountService` reuse it instead of repeating it.
- `delete_customer` runs the 404 check, then `account_repo.delete_by_customer_id`, then deletes the customer. The cascade is done in code because MongoDB has no `ON DELETE CASCADE`. The customer's transactions are deliberately kept.

`AccountService(repo, customer_service, audit_service)`

- **`create_account`** calls `get_customer` first, so an account can never point at a customer that doesn't exist.
- **`update_account`** can only change the account type, because that's all `AccountUpdate` carries.
- **`deposit`** and **`withdraw`**: find the account (404), check the amount (400), for a withdrawal check the balance (400), save the new balance, record the audit entry, return the updated account.
- **`get_premium_accounts(threshold)`** rejects a negative threshold, then converts with `Decimal(str(threshold))`. Going through `str` matters: `Decimal(1000.1)` is really `1000.0999999999999...`, but `Decimal("1000.1")` is exact.
- **`get_transactions(account_id)`** checks the account exists, then asks the audit service for its history.

`transfer` step by step

```
def transfer(self, data: TransferIn) -> Transaction:
    from_account = self.get_account(data.from_account_id)   # 404 if missing
    to_account = self.get_account(data.to_account_id)       # 404 if missing
    if from_account.id == to_account.id: ...                 # 400
    if data.amount <= 0: ...                                 # 400
    if data.amount > from_account.balance: ...               # 400
Insufficient funds
    self._repo.update_balance(from_account.id, from_account.balance -
data.amount)
    self._repo.update_balance(to_account.id, to_account.balance +
data.amount)
    return self._audit.record(TxnType.TRANSFER, data.amount,
from_account.customer_id,
                              from_account_id=from_account.id,
to_account_id=to_account.id)
```

- **Every check runs before anything changes.** A transfer that breaks a rule leaves both balances untouched.
- **Both balances change, then one audit record holds both account ids.**
- **It returns the `Transaction`**, which works as the receipt: type, amount, both accounts, and the time.

`AuditService(repo)`

- `record(...)` builds a `TransactionIn` with `timestamp=datetime.now(timezone.utc)` and saves it. Using UTC means times compare correctly no matter where the server runs.
- `list_transactions`, `get_transaction` (404) and `list_for_account` pass straight through to the repository.

8. Controllers: the routes

Controllers connect URLs to service methods. Every route body is one line; the decorator and the type hints carry all the HTTP details, and FastAPI does the work they describe.

Anatomy of one route

```
router = APIRouter(prefix="/api/accounts", tags=["Accounts"])

@router.post("/{account_id}/deposit", response_model=Account)
def deposit(account_id: int, data: AmountIn):
    return service.deposit(account_id, data.amount)
```

- **`APIRouter(prefix=..., tags=...)`** groups routes. Every route in the file starts with `/api/accounts`, and Swagger shows them under an "Accounts" heading.
- **`@router.post("/{account_id}/deposit")`** = the HTTP method and the path. `{account_id}` is a path parameter.
- **`account_id: int`** tells FastAPI to convert the URL text to a number. `/api/accounts/abc/deposit` gets a 422.
- **`data: AmountIn`** tells FastAPI to read the JSON body and validate it against that model.
- **`response_model=Account`** controls what goes back: only `Account`'s fields, in camelCase.
- **A parameter that isn't in the path is a query parameter.** `get_premium_accounts(threshold: float)` reads `?threshold=1000` from the URL.
- **`status_code=201`** on creates means "Created", and **`204`** on deletes means "done, nothing to send back". Without it, FastAPI uses 200.

Every endpoint

Method	Path	Body	Success	Errors
GET	`/api/customers`		200 list	
POST	`/api/customers`	`{name, email}`	200 customer	422
GET	`/api/customers/{id}`		200 customer	404
PUT	`/api/customers/{id}`	`{name, email}`	200 customer	404, 422
DELETE	`/api/customers/{id}`		204, accounts deleted too	404
GET	`/api/customers/{id}/accounts`		200 list	404
GET	`/api/accounts`		200 list	
POST	`/api/accounts`	`{customerId, accountType}`	201 account	404, 422
GET	`/api/accounts/premium?threshold=1000`		200 list	400
GET	`/api/accounts/{id}`		200 account	404
PUT	`/api/accounts/{id}`	`{accountType}`	200 account	404, 422
DELETE	`/api/accounts/{id}`		204	404
POST	`/api/accounts/{id}/deposit`	`{amount}`	200 account	400, 404
POST	`/api/accounts/{id}/withdraw`	`{amount}`	200 account	400, 404
GET	`/api/accounts/{id}/transactions`		200 list, newest first	404
POST	`/api/transfers`	`{fromAccountId, toAccountId, amount}`	201 transaction	400, 404
GET	`/api/audit/transactions`		200 list, newest first	
GET	`/api/audit/transactions/{id}`		200 transaction	404

Route order matters

`GET /premium` is declared *above* `GET /{account_id}`. FastAPI checks routes top to bottom. If `/{account_id}` came first, the word "premium" would be taken as an account id, fail the `int` check, and return a 422.

Two routers in one file

`account_controller.py` holds a second router, `transfer_router = APIRouter(prefix="/api/transfers")`. A transfer involves two accounts, so it gets its own URL instead of hanging off one account. `main.py` registers both routers.

9. Wiring: dependencies.py and main.py

`dependencies.py` creates every repository and service exactly once and hands each one what it needs. `main.py` creates the app and plugs in the routers. Together they are the only place where the pieces meet.

dependencies.py, the composition root

```
from db import db

customer_repo = CustomerRepository(db)
account_repo = AccountRepository(db)
audit_repo = AuditRepository(db)

audit_service = AuditService(audit_repo)
customer_service = CustomerService(customer_repo, account_repo)
account_service = AccountService(account_repo, customer_service,
audit_service)
```

- **Dependency injection:** a service never builds its own repository; it *receives* one in its constructor. `AccountService` doesn't know or care whether its repository uses MongoDB or a Python dictionary.
- **One shared copy of each object.** Every controller imports from this file, so all of them use the same repositories. This mattered in the in-memory version: two separate `UserRepository()` calls meant two separate dictionaries, and a user created through one was invisible to the other.
- **Order matters.** `audit_service` is created before `account_service`, because `account_service` needs it.
- **This is the file that changed for MongoDB.** Swapping storage meant changing what the repositories are built from (`db`). The services and controllers didn't change.

main.py

```
app = FastAPI(title="Banking API")
app.include_router(customer_router)
app.include_router(account_router)
app.include_router(transfer_router)
app.include_router(audit_router)
```

- `FastAPI(title=...)` creates the application; the title shows at the top of Swagger.
- `include_router` adds each controller's routes to the app.
- **What happens at startup:** Uvicorn imports `main`, which imports the controllers, which import `dependencies`, which imports `db`, which reads `.env` and creates the Mongo client. By the time the first request arrives, every object already exists.

FastAPI also has its own injection system, `Depends()`, which builds objects per request. A plain module was chosen here because it is simpler to read and the objects hold no per-request state.

10. The audit trail

Every successful deposit, withdrawal and transfer writes one record to the `transactions` collection, so any money movement can be traced afterwards. This answers the trainer's Priority 3 question: who, when, which accounts, how much.

What one record answers

Question	Field	Example
Which record?	`id`	4
What happened?	`type`	TRANSFER
Who did it?	`customerId`	1, the owner of the account the money came from (for a deposit, the owner of the receiving account)
How much?	`amount`	300.0
From which account?	`fromAccountId`	1
To which account?	`toAccountId`	3

Question	Field	Example
When?	`timestamp`	`2026-09-30T20:04:26Z` (UTC)

Which account fields each type fills in

Type	`fromAccountId`	`toAccountId`
DEPOSIT	empty	the account that received money
WITHDRAW	the account that lost money	empty
TRANSFER	the sender	the receiver

The three layers, as the trainer asked

- **`AuditController`** (`/api/audit/transactions` and `/api/audit/transactions/{id}`) lets you list everything or trace one record.
- **`AuditService`** builds each record (`record()`) and owns the 404 for a missing one.
- **`AuditRepository`** reads and writes the `transactions` collection.
- **`AccountService` depends on `AuditService`**, not on the audit repository. So the rule "how a record is built" lives in one place, and the account code just says what happened.
- `GET /api/accounts/{id}/transactions` gives one account's history, using the `$or` query so transfers show up for both accounts.

Design choices to mention

- **Append-only.** The audit repository has no update or delete method, so code can't rewrite history.
- **It outlives the data it describes.** Deleting a customer deletes their accounts but keeps their transactions. An audit trail that disappears with the customer would be useless for investigations.
- **Only successful movements are recorded.** The record is written after the balance changes, so a request that breaks a rule (400) leaves no record. Logging failed attempts too is a possible next step.
- **Times are in UTC**, so records from servers in different time zones still sort and compare correctly.

11. OOP concepts in the code

Almost every OOP concept from the Day 1 curriculum appears somewhere in this backend. Each row gives a real line you can point at.

Concept	Where it is	What to say
Classes and objects	`CustomerRepository`, `AccountService`, … and `dependencies.py` creating them	A class is the blueprint; `account_repo = AccountRepository(db)` makes one object from it
Constructor	`__init__` in every repository and service	Runs when the object is created and stores what it needs: `self._repo = repo`
Encapsulation / access modifiers	`self._repo`, `self._col`, `_doc_to_account`, `_options`	Python has no `private` keyword; a leading `_` means "internal, don't use from outside" (Java would say `private`)
Inheritance	`Customer(CustomerIn)`, `Transaction(TransactionIn)`, every model from `CamelModel`	The child gets every field of the parent and adds its own (`id`)
Multiple inheritance	`AccountType(str, Enum)`	It is both a string and an enum
Overriding	`DecimalCodec(TypeCodec)` overrides `transform_python` and `transform_bson`	The parent defines the methods; our class supplies its own versions
Polymorphism	The in-memory and MongoDB repositories have the same method names	Services called `find_by_id` the same way before and after the switch, so they didn't change
Abstraction	Services call `self._repo.find_premium(...)`	The service knows *what* it wants, not *how* Mongo finds it

Concept	Where it is	What to say
Composition ("has-a")	`AccountService` has a repository, a `CustomerService` and an `AuditService`	Objects built from other objects, passed in through the constructor
Static methods	`AccountIn.example()`, `AuditRepository._doc_to_transaction`	Belong to the class, need no object, so they take no `self`
Enums	`AccountType`, `TxnType`	A fixed set of allowed values
Method parameters, return types	`def get_account(self, account_id: int) -> Account:`	Type hints say what goes in and what comes out; FastAPI uses them to validate
Default parameters (Python's "overloading")	`record(..., from_account_id: int | None = None, to_account_id: int | None = None)`	Python has no method overloading; default values let one method be called in several ways

Two to be honest about

- **`final`:** Python has no `final` keyword. Nothing in this code uses `typing.Final`.
- **Interfaces:** Python has no `interface` keyword either. The two repository versions match by having the same method names ("duck typing"). An `ABC` abstract base class could make that contract explicit and is a reasonable improvement to suggest.

12. Design decisions and trade-offs

Every choice below had a real alternative. Being able to say why you picked one, and what it costs, is what makes a design defensible.

Decision	Chosen	Alternative	Why, and the cost
Web framework	FastAPI	Flask	Validation and Swagger come built in. Cost: async concepts to learn later.

Decision	Chosen	Alternative	Why, and the cost
Database	MongoDB Atlas	MySQL	Matches the trainer's task and needs no local install. Cost: the assignment mentions an SQL script, and links aren't enforced by the database.
Ids	Numbers via a `counters` collection	Mongo's own `ObjectId`	Short, readable URLs (`/api/accounts/1`) that match the assignment's examples. Routes didn't change from the in-memory version. Cost: one extra small write per insert.
Money type	`Decimal` stored as `Decimal128`	`float`	Exact: no rounding errors in balances. Cost: needed the codec in `db.py`.
JSON naming	camelCase out, snake_case in Python	One style everywhere	Each side gets its own convention (JavaScript and Python). Cost: one line of config in `CamelModel`.
Customer to accounts	Reference (`customer_id` on each account)	Embed accounts inside the customer	Accounts change and get searched on their own. Cost: two queries to show a customer with accounts.
Cascade delete	Done in the service	Database cascade	MongoDB has none, so it must be code. Cost: the developer has to remember it.
Audit records	Kept forever, never edited	Deleted with the customer	An audit trail must outlive what it describes. Cost: the collection only grows.
Input models	Separate In / Update / Out models	One model for everything	A client can't set an id or a balance. Cost: more classes.

Decision	Chosen	Alternative	Why, and the cost
Errors	Services raise `HTTPException`	Custom exceptions converted in controllers	Simple and readable. Cost: services depend on FastAPI, so they're less reusable outside a web app.
Naming	Users renamed to Customers	Keep Users	Matches the trainer's spec and the language of banking.
Secrets	`.env` file, ignored by git	Password in the code	The password never reaches GitHub.

13. Testing and the live demo

The API is checked three ways: by hand in Swagger, automatically with the Postman collection (36 requests, 65 checks, all passing), and by looking at the stored data in Atlas.

Start it

```
cd C:\Users\jeffl\Documents\citibank\backend
..\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

The Postman collection (`backend/Duong_Lam_Banking-app.postman_collection.json`) runs top to bottom and saves each new id in a variable for the next request.

Folder	What it proves
1. Customers	Full CRUD works, and a missing customer gives 404
2. Accounts	CRUD works, an account needs a real customer, a bad account type gives 422, PUT can't change the balance, one customer has many accounts
3. Money	Deposit and withdraw change the balance, overdraft and negative amounts give 400, premium accounts filter by balance, a transfer moves money both ways, and invalid transfers are refused

Folder	What it proves
4. Audit	An account's history is complete and newest first, and one transfer can be traced: who, when, both accounts, how much
5. Cleanup	Deleting a customer deletes their accounts but keeps the audit record

A 3-minute live demo in Swagger (http://127.0.0.1:8000/docs)

1. Create a customer, then two accounts for them.
2. Deposit 1500 into the first account; show the balance.
3. Withdraw 500; show the balance is 1000.
4. Try to withdraw 99999; show the 400 "Insufficient funds".
5. Call premium accounts with `threshold=1000`; the account appears (1000 counts, because the rule is "at or above").
6. Transfer 300 to the second account; show the returned transaction.
7. Open `GET /api/audit/transactions/{id}` for that transfer and read out who, when, which accounts and how much.
8. Switch to Atlas, **Browse Collections, banking**, and show the same records stored as documents.
9. Delete the customer; show their accounts are gone but the transaction is still there.

Version control tells the story of the build: `git log --oneline` shows the in-memory version (`c7e0962`), the MongoDB scaffold (`fe4cb6c`) and the finished integration (`60efed3`).

14. Known limitations and next steps

The two biggest gaps are about money safety under pressure: a transfer is two separate writes, and two requests at the same moment could both pass the balance check. Naming them yourself, with the fix, shows you understand the design rather than just the happy path.

Limitation	What could go wrong	Fix
A transfer is two separate updates	If the server crashed between them, money would leave one account and never arrive	Wrap both updates in a MongoDB multi-document transaction (`client.start_session()` and `with_transaction`). Atlas supports this
Read-then-write on balances	Two withdrawals at the same instant could both read 1000 and both succeed	One atomic update that checks and changes together: `find_one_and_update({"_id": id, "balance": {"$gte": amount}}, {"$inc": {"balance": -amount}})`
No login	Anyone who can reach the API can move anyone's money	JWT authentication (on the bonus list)
Email isn't validated or unique	Two customers can share an email, or have `"abc"` as one	Pydantic's `EmailStr` and a unique index on `customers.email`
Failed attempts aren't audited	A string of refused withdrawals leaves no trace	Record refused requests too, with a status field
Lists return everything	Slow once there are thousands of records	Pagination (`skip` and `limit`)
Services raise `HTTPException`	Rules are tied to FastAPI	Custom exceptions, converted to HTTP codes in one place
Tests live only in Postman	Nothing runs automatically on each commit	`pytest` unit tests for the services
No CORS settings	The React frontend will be blocked by the browser	`CORSMiddleware` in `main.py`, the first step of the React work

15. Likely questions, with answers

Why three layers instead of putting everything in the route? Each layer changes for a different reason. When the storage moved from dictionaries to MongoDB, only the repositories and the wiring changed; the rules and routes stayed the same.

What happens if I deposit -50? The service rejects it with 400 "Amount must be positive". Nothing is saved and no audit record is written.

And if I send `{"amount": "abc"}`? Pydantic rejects it with 422 before our code runs, because `AmountIn.amount` must be a number.

Why `Decimal` and not `float`? Floats can't store most decimals exactly: `0.1 + 0.2` is `0.30000000000000004`. `Decimal` is exact, and MongoDB stores it as `Decimal128`.

How do you get ids 1, 2, 3 in MongoDB? A `counters` collection holds one number per collection. `$inc` adds 1 in a single atomic step, so two requests can't get the same id.

How is "one customer, many accounts" stored? Each account document holds `customer_id`, like a foreign key. `find({"customer_id": 1})` returns all of that customer's accounts.

What happens to transactions when a customer is deleted? Their accounts are deleted, but the transactions stay. An audit trail must outlive the records it describes.

Can a PUT change an account's balance? No. `AccountUpdate` only has `accountType`, so any `balance` in the body is ignored. Only deposit, withdraw and transfer move money.

Why is `/premium` above `/{account_id}`? Routes are matched top to bottom. Otherwise "premium" would be read as an account id and fail with 422.

Is the transfer safe if the server crashes halfway? Not yet: it is two separate updates. The fix is a MongoDB transaction around both, which Atlas supports.

Where is the database password? In `backend/.env`, loaded by `python-dotenv`. `.gitignore` keeps it off GitHub.

What does `from_doc` do? It turns a MongoDB document into a Pydantic model, renaming Mongo's `_id` to our `id`.

What does `ReturnDocument.AFTER` mean? `find_one_and_update` returns the document after the change instead of before, so the API can send back the new balance.

What's the difference between 400, 404 and 422? 404: the thing doesn't exist. 400: it exists, but a rule says no. 422: the request itself is malformed.

How did you test it? Swagger by hand, and a Postman collection of 36 requests with 65 automated checks covering every route, every error case and the cascade delete.

How will the React frontend use this? It calls the same URLs with `fetch` and gets the same camelCase JSON. The backend only needs CORS enabled in `main.py`.